

Cahier des charges MEDIBOT

M1 ILSSEN et IA

BOUKRAA Asma

Mohamed Achraf Ouassim ARIQUI

04/01/2026

MASTER
ILSEN et INTELLIGENCE ARTIFICIELLE

UE UE PROJET D'INNOVATION

ECUE UE AMS PROJET-Interactions vocales humain-robot

Responsable
Fabrice Lefevre

Sommaire

Titre	1
Sommaire	2
1 Contexte et Objectifs du Projet	4
1.1 Contexte	4
1.2 Objectifs	4
2 Périmètre du Projet	4
2.1 Fonctionnalités attendues	4
2.2 Technologies utilisées	4
2.3 Interaction vocale avec le patient	4
2.4 Ronde nocturne proactive	5
2.5 Protocole de questions santé/confort	5
2.6 Détection d'absence de réponse et analyse vitale apparente	5
2.7 Détection d'état émotionnel / détresse	5
2.8 G	5
2.9 Notification du personnel soignant	6
2.10 Comportement en cas d'urgence critique	6
2.11 Journalisation et traçabilité	6
2.12 Limites réglementaires et éthiques	6
3 Fonctionnalités Avancées	6
3.1 Mode dégradé (Failover Mode)	6
3.2 Tableau de bord infirmier (Dashboard minimal)	6
3.3 Analyse des risques IA et gestion des erreurs	7
4 Schéma conversationnel de la ronde nocturne	8
5 Tests	9
5.1 Tests du chatbot RASA	9
5.2 Tests unitaires des actions Python	9
5.3 Tests unitaires de l'API Flask	10
5.4 Tests du module Speech-to-Text (Whisper)	10
5.5 Objectifs des tests utilisateurs	10
5.6 Méthodologie	10
6 Public Cible	10
6.1 Profil des utilisateurs	10
6.2 Nombre d'utilisateurs pour les tests	11
7 Scénarios de Tests Utilisateurs	11
7.1 Objectifs des tests	11
7.2 Scénarios de tests	11
8 Critères de Réussite et Évaluations	11
8.1 Indicateurs (KPI)	11
8.2 Feedback	11

9	Contraintes et Risques	11
9.1	Contraintes techniques	11
9.2	Risques	11
10	Planning Prévisionnel	11
10.1	Phase 1 – Développement du chatbot RASA (01/02 → 20/02)	12
10.2	Phase 2 – Implémentation du module Speech-to-Text (Whisper) (24/02 → 05/03)	12
10.3	Phase 3 – Développement robotique (Pepper, NAOqi, Choregraphe) (09/03 → 02/04)	12
10.4	Phase 4 – Détection des émotions (caméra + DeepFace/OpenCV) (06/04 → 20/04)	12
10.5	Phase 5 – Intégration complète des modules (24/04 → 01/05)	12
10.6	Phase 6 – Tests utilisateurs (05/05 → 10/05)	12
10.7	Phase 7 – Rédaction du rapport & préparation de la soutenance (10/05 → 15/05)	12
10.8	Diagramme de Gantt	13
10.9	Répartition générale des rôles	13

Sommaire

1 Contexte et Objectifs du Projet

1.1 Contexte

Dans les hôpitaux, le personnel de nuit est souvent limité et doit répondre à de nombreuses sollicitations des patients.

L'idée de MediBot est de déployer un robot humanoïde Pepper capable d'apporter un premier niveau d'interaction vocale avec les patients.

Le robot permet d'améliorer le confort, de réduire l'isolement et d'optimiser le temps du personnel soignant.

1.2 Objectifs

- Développer un chatbot vocal embarqué dans un robot humanoïde.
- Répondre à des questions simples des patients (heure, infos hospitalières, c'est quand la prochaine dose du médicament , c'est quand vous aller repasser).
- Rassurer les patients par une discussion sociale (est ce que cava ,blagues, musique, mots réconfortants).
- Alerter le personnel soignant en cas de demande urgente ou de situation anormale détectée (absence de réponse, malaise, détresse émotionnelle, arrêt respiratoire apparent).

2 Périmètre du Projet

2.1 Fonctionnalités attendues

- **Interactions vocales basiques** : répondre à « Quelle heure est-il ? », « Quel jour on est ? », « Je peux sortir quand ? »
- **Support émotionnel** : raconter une blague, proposer une musique apaisante.
- **Aide aux patients** : rappeler la prise de médicaments (base de données factice).
- **Communication** : possibilité de transmettre un message ou une alerte à un soignant.
- **Gestuelle** : hochements de tête, mouvements de bras pour humaniser l'interaction.

2.2 Technologies utilisées

- **Chatbot** : RASA, RASA-X (API REST).
- **Langage** : Python (actions, API), Flask , Pytest.
- **Robot** : Pepper, NAOqi, Choregraphe Suite 2.5.10.
- **Speech-to-text** : OpenAI Whisper.
- **Text-to-speech** : moteur vocal Pepper.
- **Base de données factice** : planning médicaments et urgences , SQLite.
- **IA** : gestion de dialogue (TEDPolicy), apprentissage par renforcement (optionnel avec TensorFlow/Keras).

2.3 Interaction vocale avec le patient

- Comprendre et répondre à des demandes simples.
- Utiliser la reconnaissance et la synthèse vocale.
- **Module** : ALTextToSpeech, micro de Pepper, RASA NLU, Flask API.

2.4 Ronde nocturne proactive

Pepper effectue périodiquement des **rondes** de nuit dans le service, il passera à chaque fois que l'aide soignant lui déclenchera une ronde.

Dialogue de check-up :

1. Présentation calme : « Bonsoir, c'est Pepper, je viens vérifier comment vous allez. »
2. Questions standardisées : douleur, respiration, stress, besoin d'aide.
3. Analyse des réponses + observation faciale.

Outils : ALNavigation, ALVideoDevice, micro, RASA NLU, NAOqi Python, planification interne (cron/timer).

2.5 Protocole de questions santé/confort

Questions posées à chaque ronde :

- « Avez-vous mal quelque part ? »
- « Avez-vous du mal à respirer ? »
- « Êtes-vous stressé(e) ou inquiet/inquiète ? »
- « Souhaitez-vous qu'une infirmière vienne maintenant ? »

Analyse automatique des réponses ; classification en trois niveaux :

RAS / à surveiller / urgence vitale.

Outils : micro, Speech-to-Text, RASA intents (douleur, détresse, urgence), script de décision Python.

2.6 Détection d'absence de réponse et analyse vitale apparente

Si le patient ne répond pas, Pepper :

1. Répète une fois la sollicitation vocale.
2. Analyse visuelle du thorax (caméra frontale) pendant 10 s : mouvement = respiration ; absence = suspicion.
3. Vérifie la posture (allongé normal / chute / penché).

Classe les cas :

- respiration normale → probable sommeil, pas d'alerte ;
- respiration absente ou posture anormale → alerte.

Outils : ALVideoDevice, OpenCV / flux optique, Python, LED/voix pour retour visuel.

Pepper ne déclare jamais un décès, il signale « urgence vitale potentielle ».

2.7 Détection d'état émotionnel / détresse

Analyse du visage et du ton de la voix pour identifier : joie, peur, douleur, angoisse.

Si incohérence entre parole et émotion → alerte moyenne ("patient anxieux").

Outils : ALFaceDetection, ALMood, ALFaceCharacteristics, caméra RGB. Micro pour prosodie. Script Python fusionnant données visuelles et sonores.

2.8 G

estion des niveaux d'alerte **Outils** : moteur de règles Python ; base SQLite locale ; commu-

Niveau	Signification	Action
0	RAS	aucune action, simple log
1	à surveiller	notification standard au personnel
2	urgence vitale potentielle	alerte immédiate + arrêt de la ronde

nication HTTP POST vers serveur infirmier.

2.9 Notification du personnel soignant

Envoi d'une alerte structurée : **chambre, heure, motif, niveau priorité.**

Modes :

- réseau local : API REST / dashboard infirmier ;
- mode dégradé : signal LED / message vocal.

Outils : Flask serveur interne, Wi-Fi sécurisé, NAOqi HTTP request.

2.10 Comportement en cas d'urgence critique

Pepper :

- reste près du lit,
- répète calmement : « Je vais chercher quelqu'un tout de suite. »,
- allume LEDs rouges,
- interrompt sa ronde.

Outils : ALMotion, ALAutonomousLife, contrôle LED, ALTextToSpeech.

2.11 Journalisation et traçabilité

Stocker pour chaque passage : heure, chambre, statut, alerte ?

Base : SQLite locale (aucune donnée médicale sensible).

Consultation par personnel autorisé uniquement.

2.12 Limites réglementaires et éthiques

- Aucune déclaration de décès.
- Aucun diagnostic médical.
- Données audio/vidéo traitées localement (RGPD).
- Annonce explicite des intentions ("Je viens vérifier votre état").
- Pas de contact physique avec le patient.
- Priorité à la sécurité et à la transparence.

3 Fonctionnalités Avancées

3.1 Mode dégradé (Failover Mode)

Afin de garantir une continuité de service dans le contexte sensible d'un service hospitalier de nuit, MediBot intégrera un **mode dégradé** permettant de maintenir des interactions essentielles même en cas de panne d'un ou plusieurs modules.

Le mode dégradé couvre les situations suivantes :

- **Panne du module Speech-to-Text (Whisper)** → Le robot passe automatiquement en mode "commandes simples" utilisant des réponses fermées (Oui / Non / Répéter).
- **Indisponibilité du module caméra** → La détection d'émotions et l'analyse respiratoire sont temporairement désactivées, sans bloquer la ronde ou le reste du dialogue.
- **Erreur du serveur de dialogue RASA** → Pepper utilise un ensemble minimal de phrases pré-enregistrées pour rassurer le patient (ex. « Je suis là si vous avez besoin de moi. »).

Objectif : Assurer que MediBot reste fonctionnel et rassurant dans toutes les situations, même en cas de défaillance partielle du système.

3.2 Tableau de bord infirmier (Dashboard minimal)

Une interface web simple (basée sur Flask) sera mise à disposition du personnel soignant afin de faciliter le suivi du robot et des alertes générées.

Le dashboard permettra de :

- visualiser les **alertes en cours** et leur niveau de criticité ;
- consulter l'**historique des rondes nocturnes**, avec heure de passage et état du patient ;
- afficher les **éventuelles émotions détectées** au cours de l'interaction ;
- suivre les **interventions déjà déclenchées**.

Avantages attendus : amélioration de la coordination entre MediBot et l'équipe soignante ; visibilité sur les événements nocturnes ; meilleure traçabilité des interactions patient-robot.

3.3 Analyse des risques IA et gestion des erreurs

Dans le cadre de l'utilisation de modules d'intelligence artificielle (RASA, Whisper, Deep-Face), une **analyse des risques liés à l'IA** sera réalisée afin d'identifier les limites techniques et leurs impacts potentiels sur la sécurité du patient.

Les principaux risques étudiés incluront :

- Erreur de compréhension vocale due au bruit ambiant nocturne ;
- Mauvaise classification d'émotion, pouvant mener à une fausse alerte ou à l'absence d'alerte ;
- Faux positifs / faux négatifs dans la détection de respiration ;
- Erreur d'interprétation du dialogue dans les intents critiques (douleur, détresse, urgence).

Des mécanismes de compensation seront intégrés, par exemple :

- confirmation systématique des phrases critiques : « Voulez-vous que j'appelle un soignant ? »
- double vérification avant une alerte niveau 2 ;
- bascule automatique en mode dégradé en cas de doute.

Objectif : Garantir un comportement fiable, sécurisant et transparent du robot dans un cadre médical.

4 Schéma conversationnel de la ronde nocturne

[DÉBUT DE RONDE NOCTURNE]

Robot : (entre dans la chambre)

- Ouvrir la porte (ALMotion + planification)
- Activer caméra et micro
- LED vertes (mode calme)

Robot : "Bonsoir, c'est Pepper, je viens vérifier comment vous allez."

Patient répond

"Ça va"

Robot : "Parfait. Souhaitez-vous que je vous chante quelque chose ou préférez-vous discuter ?"

Si "Chanson" → Robot exécute behavior musique (LED bleues)

Si "Discuter" → Robot : "Avez-vous des questions à me poser ?"

- "Quelle heure est-il ?" → réponse (via Python `datetime`)
- "Quel jour on est ?" → réponse (date)
- "Je peux sortir quand ?" → réponse générique (policy hospitalière)
- "C'est quand ma prochaine dose de médicament ?" → recherche dans BD factice
- "Peux-tu appeler mon infirmière ?" → alerte niveau 1 (Flask REST → personnel)

Si "Non merci"

Robot : "Très bien, je vous laisse vous reposer. Bonne nuit !"

- Enregistre état : patient_state = stable
- LED vertes → mode veille

Retour au début de ronde suivante (cron)

"Non, ça ne va pas"

Robot : "Souhaitez-vous que j'appelle l'équipe d'urgence ?"

Si "Oui"

Envoi d'alerte niveau 2 (urgence vitale potentielle)

- Flask HTTP POST → serveur infirmier
- LEDs rouges + message vocal :
"Je vais chercher quelqu'un tout de suite."
- Robot reste près du lit (ALAutonomousLife suspendue)

Si "Non"

Robot : "Souhaitez-vous que je vous raconte une blague ou que je vous mette une musique apaisante ?"

- Si "Oui" → Behavior play_joke ou play_music
- Si "Non" → "Très bien, reposez-vous bien."

Retour à la boucle de surveillance

[SI AUCUNE RÉPONSE APRÈS 10s]

Robot : "Je n'ai pas entendu de réponse, tout va bien ?"

Si toujours aucune réponse

- Activer analyse visuelle
 - Détection respiration (mouvement thorax)
 - Vérification posture (caméra + OpenCV)


```
Respiration normale → log "sommeil probable" (niveau 0)
Absence de respiration ou posture anormale
    Alerte niveau 2 (urgence vitale potentielle)
    LEDs rouges + message vocal :
    "Je vais chercher quelqu'un tout de suite."
```

Fin de séquence / journalisation

5 Tests

Cette section décrit la stratégie de tests mise en place pour le projet **MediBot**. Les tests portent exclusivement sur les **modules logiciels**, indépendamment du robot physique Pepper, afin de valider la logique, la robustesse et le bon fonctionnement du système.

Les tests unitaires sont réalisés à l'aide de données simulées, de composants mockés et de scénarios contrôlés, conformément aux bonnes pratiques en ingénierie logicielle et en systèmes embarqués.

5.1 Tests du chatbot RASA

Les tests RASA permettent de vérifier le bon fonctionnement du moteur conversationnel, aussi bien au niveau de la compréhension du langage naturel que de la gestion des dialogues.

5.1.1 Éléments testés

- Compréhension des intentions (intents).
- Extraction correcte des entités.
- Déclenchement de la bonne réponse ou de la bonne action.

5.1.2 Tests NLU

Les tests NLU permettent d'évaluer :

- Le taux de reconnaissance des intentions.
- La confusion entre intentions proches (ex. discussion sociale vs demande urgente).

Ces tests sont réalisés à partir d'exemples annotés et mesurés automatiquement à l'aide des outils fournis par RASA.

5.1.3 Tests Core (gestion du dialogue)

Les tests du moteur de dialogue permettent de vérifier que :

- Une phrase exprimant une urgence déclenche correctement une action d'alerte.
- Une discussion sociale ou rassurante ne déclenche aucune alerte.

Ces tests valident la cohérence des stories, des règles conversationnelles et du comportement global du chatbot.

5.2 Tests unitaires des actions Python

Les actions personnalisées implémentées en Python sont testées indépendamment du chatbot afin de garantir la fiabilité de la logique métier.

5.2.1 Éléments testés

- Logique métier des actions.
- Règles de décision.
- Calcul et attribution des niveaux d'alerte.

Les tests permettent notamment de vérifier que les différentes combinaisons de signaux (douleur, émotion, respiration, absence de réponse) conduisent au niveau d'alerte approprié.

5.3 Tests unitaires de l'API Flask

L'API Flask, utilisée pour la gestion des alertes et le tableau de bord infirmier, fait l'objet de tests unitaires spécifiques.

5.3.1 Éléments testés

- Fonctionnement des endpoints REST.
- Format et structure des alertes transmises.
- Codes de réponse HTTP (succès, erreur).

Ces tests garantissent la fiabilité des échanges entre MediBot et les systèmes de supervision du personnel soignant.

5.4 Tests du module Speech-to-Text (Whisper)

Important : la qualité acoustique réelle n'est pas évaluée dans les tests unitaires. Les tests portent uniquement sur le **pipeline de transcription**.

5.4.1 Éléments testés

- Transformation d'un fichier audio pré-enregistré en texte.
- Gestion des erreurs (fichier manquant, audio corrompu).
- Temps de réponse acceptable pour un usage en contexte hospitalier.

5.5 Objectifs des tests utilisateurs

- Vérifier la compréhension de la voix dans un environnement bruyé (nuit, couloir).
- Valider la pertinence des réponses.
- Tester la capacité du robot à rassurer l'utilisateur.

Ces tests permettent de s'assurer que le module Whisper s'intègre correctement dans la chaîne de traitement du dialogue.

5.6 Méthodologie

La méthodologie de test du projet MediBot repose sur deux approches complémentaires :

- **Tests modérés** : des étudiants jouent le rôle de patients, permettant une observation encadrée des interactions et une analyse détaillée du comportement du robot.
- **Tests non modérés** : le robot est laissé à disposition et fonctionne de manière autonome, afin d'évaluer son comportement dans des conditions d'utilisation proches du réel.

6 Public Cible

6.1 Profil des utilisateurs

- Patients hospitalisés (personnes âgées ou adultes, faible compétence numérique).
- Personnel soignant (infirmiers, aides-soignants).

6.2 Nombre d'utilisateurs pour les tests

- Phase exploratoire : 3–4 testeurs (simulation étudiants).
- Phase de validation : 6–8 testeurs (simulation + encadrants).

7 Scénarios de Tests Utilisateurs

7.1 Objectifs des tests

- Vérifier la compréhension de la voix dans un environnement bruyé (nuit, couloir).
- Valider la pertinence des réponses.
- Tester la capacité du robot à rassurer l'utilisateur.

7.2 Scénarios de tests

- Demander l'heure actuelle.
- Demander un rappel de médicament.
- Demander à appeler un soignant.
- Demander une blague ou une musique apaisante.

8 Critères de Réussite et Évaluations

8.1 Indicateurs (KPI)

- Taux de réussite : >60 %.
- Temps de réponse moyen < 15 secondes.
- Satisfaction utilisateur $\geq 3/5$.

8.2 Feedback

- Questionnaires Likert 1–5 (compréhension, réassurance, utilité).
- Interviews rapides avec les testeurs.

9 Contraintes et Risques

9.1 Contraintes techniques

- Latence cloud (Google Speech).
- Disponibilité limitée du robot.
- Base de données fictive pour tests .

9.2 Risques

- Bruit ambiant perturbant la reconnaissance vocale.
- Bugs d'intégration RASA ↔ Pepper.
- Retard dans la mise en place des scénarios réalistes.

10 Planning Prévisionnel

Le développement du projet MediBot s'étend sur la période **du 1er février au 15 mai**, correspondant à la durée du semestre 2.

Le planning est organisé en **phases des sprints successives**, chacune suivie de **tests unitaires**

obligatoires (2 à 3 jours) afin d'assurer la qualité avant de poursuivre l'intégration. Le diagramme de Gantt ci-dessous présente la planification détaillée de l'ensemble du projet.

10.1 Phase 1 – Développement du chatbot RASA (01/02 → 20/02)

- Création des intents, entités, stories et règles.
- Implémentation des actions Python (médicaments, alertes, émotions).
- Tests unitaires : 21/02 → 23/02

10.2 Phase 2 – Implémentation du module Speech-to-Text (Whisper) (24/02 → 05/03)

- Capture audio, transcription locale Whisper.
- Envoi automatique des messages transcrits vers l'API REST RASA.
- Tests unitaires : 06/03 → 08/03

10.3 Phase 3 – Développement robotique (Pepper, NAOqi, Choregraphe) (09/03 → 02/04)

- Voix du robot (TTS), mouvements, gestes apaisants.
- Contrôle des LEDs et comportements contextuels.
- Scripts NAOqi et scénarios Choregraphe.
- Tests unitaires : 03/04 → 05/04

10.4 Phase 4 – Détection des émotions (caméra + DeepFace/OpenCV) (06/04 → 20/04)

- Analyse du visage et du ton de la voix.
- Détection de tristesse, stress, douleur, inquiétude.
- Intégration avec les slots RASA.
- Tests unitaires : 21/04 → 23/04

10.5 Phase 5 – Intégration complète des modules (24/04 → 01/05)

- Chaîne complète : patient → Whisper → RASA → robot → base de données → alerte.
- Intégration du scénario de ronde nocturne.
- Tests d'intégration : 02/05 → 04/05

10.6 Phase 6 – Tests utilisateurs (05/05 → 10/05)

- Patients simulés (étudiants).
- Validation de la compréhension vocale, de la pertinence des réponses, et du niveau de réassurance du robot.

10.7 Phase 7 – Rédaction du rapport & préparation de la soutenance (10/05 → 15/05)

- Rapport complet (méthodes, résultats, architecture).
- Préparation de la présentation finale.
- Vidéo démonstrative du robot en situation réelle.

10.8 Diagramme de Gantt

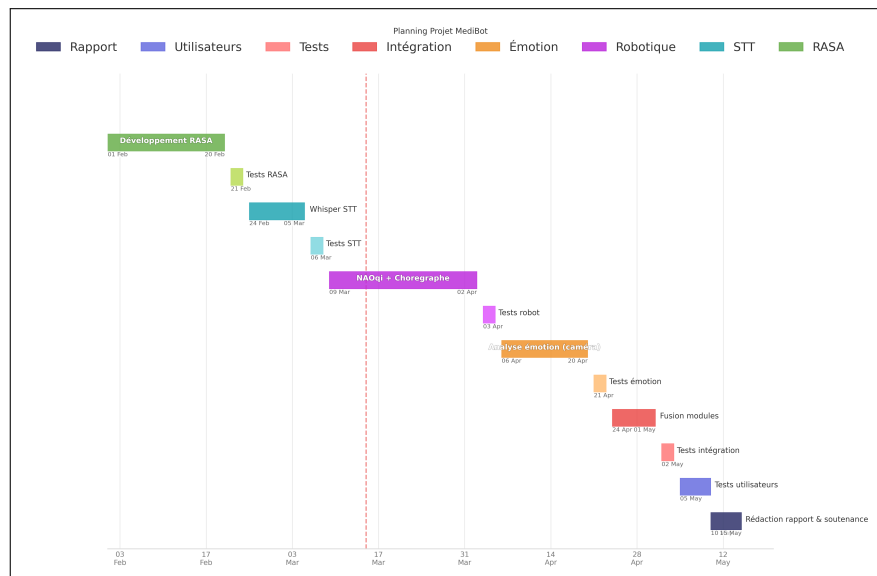


Figure 1. Diagramme de gantt qui contient les phases de dev + les tests

Le diagramme de Gantt illustre l'ensemble des phases, leurs durées, leurs dépendances et les périodes de tests unitaires.

10.9 Répartition générale des rôles

BOUKRAA Asma : intervient sur le développement du système conversationnel et vocal, la logique décisionnelle, l'intégration des modules logiciels ainsi que la mise en place et l'analyse des tests.

ARIOUI Mohamed Achraf Ouassim : intervient sur l'intégration robotique de Pepper, les comportements physiques et visuels, la communication avec les modules logiciels, ainsi que la démonstration, la validation finale et la présentation du projet.

Les choix d'architecture, l'intégration globale, les tests finaux et la validation du projet sont réalisés conjointement.